國 立 清 華 大 學

National Tsing Hua University

碩 士 論 文

Master's Thesis

分散式AI 代理系統中過時工作失效之隨機分析

Stochastic Analysis of Stale-Work Invalidation in Distributed AI-Agent Systems

系所別： 通訊工程研究所

Dept. / Grad. Inst.: Institute of Communications Engineering

學號 Student ID: 113064511

研究生 Author： 陳品仲 Pin-Chung Chen

指導教授 Advisor： 張正尚 Prof. Cheng-Shang Chang

中 華 民 國 115 年 6 月

June 2026

國 立 清 華 大 學

National Tsing Hua University

碩 士 論 文

Master's Thesis

分散式AI 代理系統中過時工作失效之隨機分析

Stochastic Analysis of Stale-Work Invalidation in Distributed AI-Agent Systems

系所別： 通訊工程研究所

Dept. / Grad. Inst.: Institute of Communications Engineering

學號 Student ID: 113064511

研究生 Author： 陳品仲 Pin-Chung Chen

指導教授 Advisor： 張正尚 Prof. Cheng-Shang Chang

中 華 民 國 115 年 6 月

June 2026

學位論文授權書

.

.

指導教授推薦書

.

.

考試委員審定書

.

.

摘要

分散式人工智慧代理系統日益依賴共享狀態，例如持久記憶、資料庫、文件、程式碼儲存庫與環境狀態。在樂觀式共享狀態執行中，一次嘗試可能先讀取某個狀態版本，接著進行長時間計算，最後在提交時因爲期間已有重疊嘗試成功更新共享狀態而被拒絕。此類陳舊工作失效會浪費計算並引發重試。

本文建立一個隨機拒絕與重試框架來分析此機制。模型中，外部任務到達會產生首次嘗試；被拒絕的嘗試會在隨機重啓延遲後產生重試；每一次成功提交會以機率 α 干擾重疊執行中的嘗試。本文提出 Poisson offered-load approximation，將首次嘗試與重試嘗試的總流量表示爲速率 G 的平穩 Poisson 過程。對外部任務到達率 λ，操作點 $G^\star$ 由速率平衡方程 $S_\alpha(G^\star) = \lambda$ 決定，其中 $S_\alpha(G)$ 爲 offered load 爲 G 時的成功提交率。

在完全干擾情形 $\alpha = 1$ 下，本文建立成功提交時刻的更新過程刻畫，並推導一般嘗試處理時間分布的吞吐量表示式，以及確定性與指數分布的明確特例。在部分干擾情形 $0 \le \alpha < 1$ 下，本文提出針對存活執行中嘗試的年齡式平均場近似，並推導相應的固定點方程。

離散事件模擬驗證分析趨勢，顯示干擾機率是提交有效機率、重試放大與完成延遲的關鍵驅動因素。結果指出，改善共享狀態 AI 代理系統的可擴展性，應重視降低陳舊工作失效，而不只是增加計算資源。

關鍵詞： 分散式人工智慧代理系統、樂觀式並行控制、重試放大、共享狀態、隨機模型。

Abstract

Distributed AI-agent systems increasingly rely on shared state, such as persistent memory, databases, documents, code repositories, and environment states. In optimistic shared-state execution, an attempt may read a state version, perform a long computation, and then be rejected at commit time because a successful overlapping attempt has made its work stale. Such stale-work invalidation wastes computation and induces retries. This paper develops a stochastic reject-and-retry framework for analyzing this mechanism. We formulate a model in which external task arrivals generate first attempts, rejected attempts generate retries after random restart delays, and successful commits disrupt overlapping in-flight attempts with probability α. We then develop a Poisson offered-load approximation in which the aggregate stream of first and retry attempts is represented by a stationary Poisson process of rate G. For an external task-arrival rate λ, the operating point $G^\star$ is selected by the rate-balance equation $S_\alpha(G^\star) = \lambda$, where $S_\alpha(G)$ is the successful-commit rate under offered load G. For the complete-disruption case $\alpha = 1$, we obtain a renewal characterization of successful-commit epochs and derive throughput expressions for general attempt-processing-time distributions, with explicit deterministic and exponential special cases. For the partial-disruption case $0 \leq \alpha < 1$, we introduce an age-based mean-field approximation for surviving in-flight attempts and derive the corresponding fixed-point equations. Discrete-event simulations confirm the analytical trends and show that the disruption probability is a key driver of commit-validity probability, retry amplification, and completion delay. These results provide a quantitative basis for improving the scalability of shared-state AI-agent systems by reducing stale-work invalidation, rather than relying only on additional compute capacity.

Keywords: Distributed AI-agent systems, optimistic concurrency control, retry amplification, shared state, stochastic modeling.

致謝

The author would like to thank Prof. Don Towsley for his valuable comments during the writing of this thesis. This work was supported in part by the National Science and Technology Council, Taiwan, under Grant NSTC 114-2221-E-007-067-MY3.

目　次

學位論文授權書

指導教授推薦書

考試委員審定書

摘要 **i**

Abstract **ii**

致謝 **iii**

目次 **v**

圖次 **vi**

表次 **vii**

1 Introduction **1**

2 Related Work **6**

3 System Model and Problem Formulation **8**

3.1 External Tasks and Attempt Dynamics 8

3.2 Commit-Time Validity and Disruption 9

3.3 Offered-Load Approximation 10

4 Performance Analysis for Complete Disruption **13**

4.1 Renewal Analysis 13

4.1.1 Deterministic Attempt Processing Time 15

4.1.2 Exponential Attempt Processing Time 16
4.2 Mean Completion Delay 17
4.2.1 Finite-G Operating Region 18

5 Performance Analysis for Partial Disruption 21
5.1 Mean-Field Throughput Analysis 21
5.2 Mean Completion Delay 27
5.2.1 Finite-G Operating Region 27

6 Simulation Results 31
6.1 Simulation Setup 31
6.2 Aggregated Attempt Rate Versus External Task-Arrival Rate 33
6.3 Commit-Validity Probability Versus External Task-Arrival Rate 33
6.4 Mean Completion Delay Versus External Task-Arrival Rate 34
6.5 Discussion 35

7 Conclusion 38

參考文獻 41

圖　次

1-1 Stale-work invalidation in a distributed multi-agent shared-state system. Agent A commits an update to the shared state while Agent B is still computing from an older version. When Agent B later attempts to commit, its update is stale; the attempt is rejected, and the task must be retried. 2

4-1 Renewal structure in the complete-disruption regime. A successful commit at epoch τ disrupts all in-flight attempts; the next successful commit must therefore be generated by a new attempt started after τ. 14

5-1 Space-time execution timeline for the partial-disruption regime. The next successful commit can arise either from a pre-τ in-flight attempt or from a post-τ attempt. The depicted pre-τ in-flight attempt is schematic: in the mean-field approximation, its retention is age dependent and is represented by $r(a;S)$ in (5.1). 23

6-1 Aggregated attempt rate G versus external task-arrival rate λ for (a) deterministic and (b) exponential attempt processing times. Dashed curves show analytical predictions; solid curves with markers show simulation. . 33

6-2 Commit-validity probability $p_v = S/G$ versus external task-arrival rate λ for (a) deterministic and (b) exponential attempt processing times. Dashed curves show analytical predictions; solid curves with markers show simulation. 34

6-3 Mean completion delay $\mathbb{E}[W]$ versus external task-arrival rate λ for (a) deterministic and (b) exponential attempt processing times. Dashed curves show analytical predictions; solid curves with markers show simulation. . 35

表　次

6-1　Median simulation error for representative values of α. 36

Chapter 1

Introduction

Distributed artificial intelligence (AI) workloads increasingly involve large language model (LLM)-based agents that communicate, invoke tools, interact with external environments, maintain memory, and coordinate through shared artifacts or state. This design pattern has roots in classical blackboard systems [1, 2] and has reappeared in contemporary LLM-based multi-agent systems [3, 4]. Several systems exemplify this trend: agents may collaborate through role-based communication, software artifacts, standardized workflows, tool calls, data-analysis environments, and web interfaces [5–8].

Persistent memory and shared context further increase the importance of shared-state coordination. Recent surveys and memory-specific studies show that memory writing, abstraction, retrieval, and adaptation are central components of agent construction, but remain nontrivial design challenges [9–11]. These features make coordination performance-sensitive because long reasoning or tool-use trajectories may depend on state observed much earlier in execution.

This paper abstracts such long-horizon, tool-using agent workflows as an optimistic read–compute–commit pattern over shared state. An agent reads the current shared state, performs a potentially long computation, and later submits a commit request. The computation may include reasoning, retrieval, tool use, planning, code execution, interaction with external environments, and update generation [12–14]. Since multiple agents may execute concurrently, the state observed at the beginning of one computation may be modified by another agent before the first computation submits its commit request. The resulting request may then be stale and, under a reject-and-retry consistency policy, is rejected to preserve the shared state.

We now introduce the main terms used throughout the paper. An *external task* is a user request, workflow item, or agent job that must eventually produce one accepted update to the shared state. An *attempt* is one speculative execution trajectory of such a task:

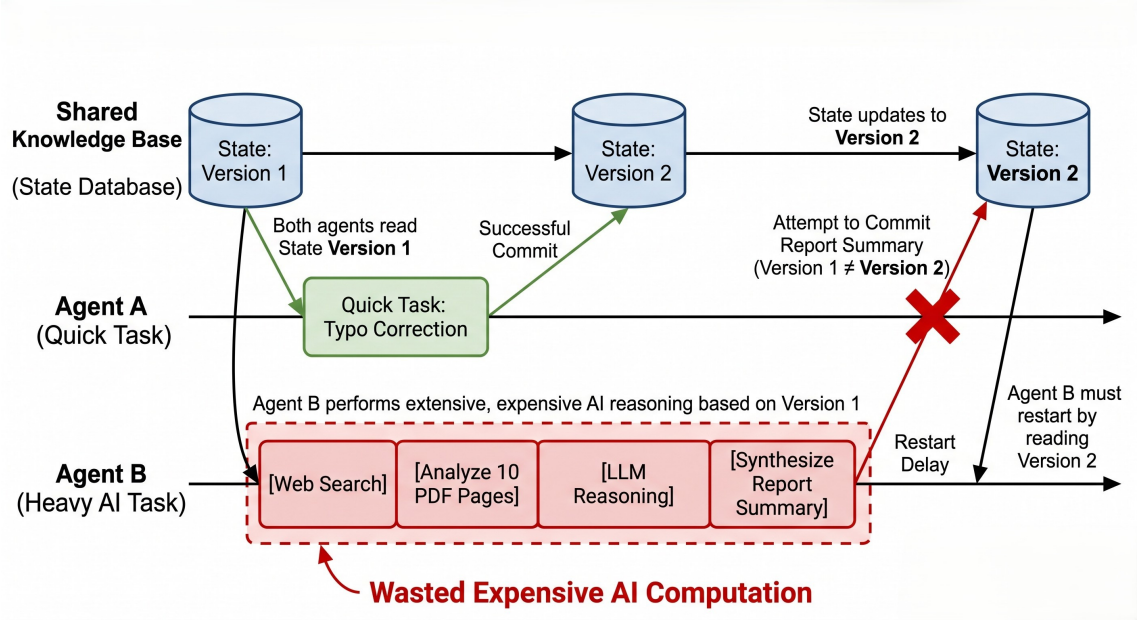


Figure 1-1: Stale-work invalidation in a distributed multi-agent shared-state system. Agent A commits an update to the shared state while Agent B is still computing from an older version. When Agent B later attempts to commit, its update is stale; the attempt is rejected, and the task must be retried.

it reads the shared state, performs computation based on the observed state, and later submits a commit request. An attempt is called *in flight* from its start time until its commit request epoch. If an intervening successful commit invalidates assumptions used by an in-flight attempt, the attempt becomes stale. In the model considered here, a stale attempt is not preempted immediately; it continues running until its own commit request epoch, at which point the request is rejected and the task retries after a restart delay. Thus, a task may require several attempts before one of them commits successfully.

Fig. 1-1 illustrates this mechanism. Two agents read the same state version. Agent A commits first, changing the shared state. Agent B is still computing from the older version and therefore submits a stale commit request later. That request is rejected, and Agent B must retry the task based on a refreshed state. Although the example contains only two agents, the same mechanism becomes a system-level concern when many long-running attempts overlap and rejected attempts return as retries.

In these workflows, staleness is not merely a version-number mismatch. A shared-state update may invalidate assumptions made during retrieval, planning, tool invocation,

code generation, or intermediate reasoning. A rejected attempt therefore wastes completed computation and generates retry traffic. This creates an amplification mechanism: external arrivals generate first attempts, rejected attempts generate retry attempts, and retry attempts increase the aggregated attempt rate experienced by the shared-state system. Consequently, the external task-arrival rate may substantially understate the actual amount of speculative computation launched by the system.

A central parameter in our model is the *per-successful-commit disruption probability*. At each successful commit epoch, each overlapping in-flight attempt that started before that commit is independently disrupted with probability α. A disrupted attempt is marked stale and will be rejected when it later submits its own commit request. Thus, α is not the probability that an arbitrary attempt is rejected. Rather, it is an effective shared-state coupling parameter: large α represents broad shared-state updates that invalidate many overlapping execution trajectories, whereas small α represents modular updates, fine-grained read/write boundaries, or weak dependence among state components. The case $\alpha = 1$ is the *complete-disruption regime*, in which every successful commit disrupts all earlier in-flight attempts. The case $0 \leq \alpha < 1$ is the *partial-disruption regime*, in which some in-flight attempts may survive intervening successful commits. The case $\alpha = 0$ corresponds to no disruption.

Although this mechanism uses primitives familiar from optimistic concurrency control, retrial queues, negative-customer queues, and restart-based execution [15, 16], existing models typically capture only neighboring aspects of the problem, such as protocol validation, open-loop rejection, exogenous clearing, or hardware-level conflict and rollback. To the best of our knowledge, prior analytical models have not jointly captured, in the context of distributed shared-state AI-agent execution, stale invalidation of overlapping agent computations, retry after failed commit, and a self-consistent aggregated attempt rate.

This paper develops a stochastic reject-and-retry framework for analyzing this mechanism. External tasks arrive at rate λ. Because a rejected task may retry until it eventually commits, the load experienced by the shared-state system is not fully described by λ. The total load is the aggregated attempt rate G, which includes both first attempts and retry attempts. The ratio G/λ is the retry-amplification factor. We also study the commit-validity

probability, defined as the probability that an arbitrary attempt remains compatible with the current shared state when it submits its commit request. These quantities are coupled: retries increase G, a larger G increases overlap among attempts, and greater overlap increases the probability that an attempt becomes stale before it commits.

The model focuses on the interaction among attempt starts, commit-time validity, rejection, retry, and task completion. Attempt processing times are random, and rejected attempts restart after random delays. Completion delay is measured from the external arrival epoch of a task to the epoch of its first successful commit; it therefore includes the processing times of all attempts made by the task and the restart delays following rejected attempts.

The analysis proceeds through the following approximation hierarchy. The true retry process is endogenous, because retry epochs depend on previous processing times, commit outcomes, disruption events, and restart delays. To obtain a tractable offered-load description, we approximate the aggregate stream of first and retry attempts by a stationary Poisson process of rate G. For each prescribed G, the Poisson offered-load approximation characterizes the successful-commit rate $S_\alpha(G)$ and the commit-validity probability. For an external task-arrival rate λ, the operating point is then selected by the rate-balance equation $S_\alpha(G) = \lambda$. In the complete-disruption regime, i.e., $\alpha = 1$, each successful commit disrupts all earlier in-flight attempts, which yields a renewal characterization of successful-commit epochs. In the partial-disruption regime, i.e., $0 \leq \alpha < 1$, some in-flight attempts may survive intervening successful commits and later complete successfully. This survival destroys the renewal structure, and we therefore use an age-based mean-field approximation to obtain a fixed-point characterization.

The main contributions are as follows:

- We formulate a stochastic reject-and-retry model for optimistic shared-state AI-agent execution. The model distinguishes the external task-arrival rate λ from the aggregated attempt rate G, and incorporates random attempt processing times, random restart delays, and a per-successful-commit disruption probability α. Within the Poisson offered-load approximation, the operating point corresponding to λ is determined by the rate-balance equation $S_\alpha(G) = \lambda$.

- For the complete-disruption regime $\alpha = 1$, we derive a renewal characterization of successful-commit epochs. This characterization yields throughput expressions for general attempt-processing-time distributions, as well as explicit deterministic and exponential special cases. For the deterministic case $X \equiv D$, where D is the attempt processing time and hence also its mean, the analysis identifies $\lambda D < 1$ as the condition under which the system admits a finite offered-load operating point.

- For the partial-disruption regime $0 \leq \alpha < 1$, we develop an age-based mean-field approximation for surviving in-flight attempts. The resulting fixed-point characterization accounts for both attempts that start after a commit epoch and attempts that survive intervening successful commits. The analysis shows that, under this approximation, any attempt-processing-time distribution with finite mean admits a finite offered-load operating point for every finite external task-arrival rate in the partial-disruption regime.

- We characterize the completion delay induced by stale-work invalidation and retry. The analysis relates the commit-validity probability, the mean number of attempts per completed task, and the mean restart-delay overhead to the mean completion delay $\mathbb{E}[W]$. This connects the throughput operating point $G^\star$ to task-level delay performance.

- We validate the analytical predictions through discrete-event simulations that explicitly generate external arrivals, retry attempts, successful commits, and disruptions of overlapping attempts. The simulations quantify how the disruption probability α controls the aggregated attempt rate, commit-validity probability, retry amplification, and completion delay under deterministic and exponential attempt-processing-time distributions.

The rest of the thesis is organized as follows. Chapter 2 reviews related work. Chapter 3 presents the model and formulation. Chapter 4 analyzes complete disruption. Chapter 5 develops the partial-disruption mean-field approximation. Chapter 6 reports simulation results, and Chapter 7 concludes the thesis.

Chapter 2

Related Work

The challenge of stale-work invalidation in shared-state systems lies at the intersection of modern LLM-agent architectures, optimistic concurrency control, and queueing models with retry or removal mechanisms. Recent LLM-based agent frameworks show that agents increasingly coordinate through conversation, shared context, persistent memory, tool-use trajectories, software artifacts, workflows, data-analysis environments, and web interfaces [5–8, 17]. Surveys organize these systems around agent construction, communication, memory, interaction, and evolution [3, 4, 18]. Persistent memory, tool use, and long-horizon environment interaction further motivate state-dependent agent execution [9–14, 19, 20]. These works motivate the workload abstraction studied in this paper, but they do not provide a stochastic model for the execution delay and workload amplification caused by stale-work invalidation.

Optimistic concurrency control (OCC) provides the closest conceptual foundation for commit-time validation. Classical OCC allows transactions to execute speculatively and validates them at commit time rather than serializing them before execution [15]. Distributed concurrency-control and multiversion models characterize how transaction ordering, validation, and read/write versions affect correctness [21,22]. Later work clarified the gap between practical isolation levels and serializability [23], formalized generalized weak isolation levels [24], and studied conditions under which snapshot isolation can be made serializable [25]. Performance studies of OCC and related transactional protocols analyze validation, contention, restart, communication, and repair costs [26–33]. These studies provide the validation and restart background for the present paper, but they primarily address database transactions and protocol-level conflict behavior rather than retry-amplified computational load in long-running shared-state AI-agent attempts.

Classical queueing and rollback models provide related mathematical primitives. Retrial queues model customers that return after blocking or rejection [16], while clear-

ing queues, negative-customer queues, and workload-clearing models describe systems in which external events remove customers or workload [34–36]. Rollback systems and transactional-memory models quantify execution under conflicts, aborts, and retries [37–40]. These models capture retry, removal, rollback, and restart effects, but they do not directly model the feedback mechanism considered here: successful commits disrupt overlapping speculative computations, rejected attempts return as retry traffic, and the resulting aggregated attempt rate affects future commit-validity probabilities. Thus, existing work provides the architectural motivation, validation mechanisms, and queueing primitives needed for our study, but it does not combine these ingredients into a stochastic model of retry-amplified stale-work invalidation in shared-state AI-agent systems.

Chapter 3

System Model and Problem Formulation

This section defines the stochastic model used in the analysis. The model isolates three core mechanisms: optimistic execution over shared state, commit-time validity checking, and retry after rejection. The analysis focuses on the shared-state commit-validity mechanism induced by overlapping attempts. Thus, the completion delay studied below includes attempt processing and retry overhead.

In the LLM-agent interpretation, the shared state represents a persistent state object accessed by multiple agents, such as shared memory, a database, a shared document, a knowledge base, or an environment state. An external task represents a user request, workflow item, or agent job that must eventually produce one accepted update to this shared state. An attempt represents one execution trajectory of that task based on the state observed at its start time.

The attempt processing time captures the latency of reasoning, retrieval, tool invocation, code execution, interaction with external environments, and formulation of the proposed update before commit validation. The commit request represents the attempt to write the generated update back to the shared state. If intervening successful commits invalidate assumptions made during the attempt, the commit request is rejected. The task then incurs a restart delay, representing context refresh, replanning, relaunch overhead, or tool-chain reinitialization before a retry.

3.1 External Tasks and Attempt Dynamics

External tasks arrive according to a Poisson process of rate $\lambda > 0$. Each task remains in the system until one of its attempts successfully commits an update to the shared state. A task may therefore require multiple attempts before completion, but it has at most one active attempt at any time.

Let $A_i^{(k)}$ denote the k-th attempt of task i, and let $s_i^{(k)}$ be its start time. The attempt runs for a random attempt processing time $X_i^{(k)}$ and issues its commit request at

$$c_i^{(k)} = s_i^{(k)} + X_i^{(k)}. \tag{3.1}$$

The attempt processing times $\{X_i^{(k)}\}$ are independent, identically distributed, nonnegative random variables with cumulative distribution function F_X. We assume $F_X(t) = 0$ for $t < 0$, write $\bar{F}_X(t) = 1 - F_X(t)$, and assume finite mean $D = \mathbb{E}[X_i^{(k)}]$. Thus, $X_i^{(k)}$ is the processing time of one speculative attempt before commit validation.

If an attempt is rejected, the corresponding task does not depart. After the rejection of $A_i^{(k)}$, task i waits for a random restart delay $B_i^{(k)} \geq 0$ and then starts the next attempt at

$$s_i^{(k+1)} = c_i^{(k)} + B_i^{(k)}. \tag{3.2}$$

The restart delays are independent of the attempt processing times and have finite mean $\bar{B} = \mathbb{E}[B_i^{(k)}]$. A zero restart delay corresponds to immediate retry. A positive restart delay captures backoff, relaunch overhead, context refresh, replanning, reinitialization of the tool-use sequence, or other retry overhead before the next speculative attempt begins.

3.2 Commit-Time Validity and Disruption

Each attempt reads the shared state at its start time and is evaluated for validity only at its commit request epoch. If the attempt remains compatible with the current shared state, the commit is accepted, the shared state is updated, and the task departs. If intervening successful commits have made the attempt stale, the commit request is rejected and the task retries according to (3.2).

Only successful commits can disrupt in-flight attempts. At each successful-commit epoch, every in-flight attempt that started before the commit is disrupted independently with probability $\alpha \in [0, 1]$, with disruption events independent across attempts and successful-commit epochs. A disrupted attempt is marked stale but is not preempted. It continues execution until its own commit request epoch, at which point the request is rejected. This

captures systems in which an agent does not continuously validate all shared-state dependencies during execution; instead, stale dependence is detected when the agent submits its proposed update.

Thus, α is a per-successful-commit disruption probability and an effective shared-state coupling parameter; it is not the probability that an arbitrary attempt is rejected. Large α corresponds to updates with broad shared-state impact, such as global memory rewrites, coarse-grained document updates, or changes that disrupt many overlapping execution trajectories. Small α corresponds to modular updates, fine-grained read/write boundaries, or weak dependence among the state components used by overlapping attempts.

If an attempt is unfinished while m successful commits occur after its start time and before its own commit request epoch, then, under the independent disruption model, the probability that the attempt survives all m intervening commits is $(1-\alpha)^m$. An attempt that survives until its commit request epoch commits successfully. Otherwise, it is rejected and the task retries.

The parameter α determines the disruption regime. When $\alpha = 1$, every successful commit disrupts all in-flight attempts that started earlier. This complete-disruption regime represents a fully coupled shared-state benchmark. The case $0 \leq \alpha < 1$ represents the partial-disruption regime, in which some in-flight attempts that started earlier may remain valid after one or more intervening successful commits.

3.3 Offered-Load Approximation

In our model, external arrivals generate first attempts, while rejected attempts generate retry attempts. Retry epochs depend on previous attempt processing times, commit outcomes, disruption events, and restart delays. Consequently, although the external task-arrival process is Poisson, the retry attempt process is generally history dependent.

For tractability, our analysis uses a Poisson offered-load approximation. The aggregate stream of first attempts and retry attempts is approximated by a stationary Poisson process of rate G, while attempt processing times remain independent and identically distributed according to F_X. For the aggregated attempt rate G, this approximation characterizes the shared-state commit-validity mechanism and yields a throughput map $S_\alpha(G)$. We note

that this type of offered-load abstraction is classical in random-access analysis, beginning with the ALOHA model, where the aggregate stream of new packets and retransmissions is represented by a Poisson offered load [41].

For a specific external task-arrival rate λ, the corresponding aggregated attempt rate $G^\star$ is obtained by solving the rate-balance equation, i.e., the departure rate equals the arrival rate,

$$S_\alpha(G^\star) = \lambda. \tag{3.3}$$

A finite-G operating point is a finite solution $G^\star < \infty$ of (3.3). The finite-G operating region is the set of external task-arrival rates λ for which such a solution exists, and it may be viewed as a weak notion of stability in classical queueing theory.

Under the Poisson offered-load approximation, let $S_\alpha(G)$ denote the successful-commit rate (i.e., the throughput) and let $p_{v,\alpha}(G)$ denote the probability that an arbitrary attempt is valid at its commit request epoch. Since each valid attempt produces exactly one successful commit,

$$S_\alpha(G) = G\, p_{v,\alpha}(G). \tag{3.4}$$

The retry-amplification factor is defined at a finite-G operating point as

$$R(\lambda, \alpha) \triangleq \frac{G^\star}{\lambda} = \frac{1}{p_{v,\alpha}(G^\star)}, \tag{3.5}$$

where $G^\star$ satisfies (3.3). It measures the amount of aggregated attempt traffic incurred by the external task traffic.

Let W denote the completion delay of a typical task under the Poisson offered-load approximation, measured from its external arrival epoch to the epoch of its first successful commit. Under this abstraction, the first attempt is launched immediately upon external arrival. Hence, W consists of the processing times of all attempts launched by the task, together with the restart delays incurred after rejected attempts.

The Poisson offered-load approximation replaces the aggregated attempt process by a Poisson process of rate G. Consequently, restart delays enter the analytical delay expression only through their mean $\bar{B}$. Their effect on retry-timing correlations is not captured by the throughput map $S_\alpha(G)$. This separation is intentional: the throughput analysis iso-

lates the commit-validity mechanism induced by overlapping attempts, whereas the delay analysis accounts for the mean retry overhead incurred by rejected tasks.

When α is fixed or clear from context, we abbreviate $S_\alpha(G)$ and $p_{v,\alpha}(G)$ as $S(G)$ and $p_v(G)$, respectively. In the analytical sections that follow, all theoretical statements concerning throughput, commit-validity probability, delay, and finite-G operating behavior are understood with respect to this Poisson offered-load approximation unless explicitly stated otherwise.

Chapter 4

Performance Analysis for $\alpha = 1$

This chapter conducts performance analysis for the complete-disruption setting, i.e., $\alpha = 1$. Accordingly, we use the shortened notation $S(G)$ and $p_v(G)$ introduced in Section 3.3, rather than writing the subscript $\alpha = 1$ explicitly. Conditioned on the Poisson offered-load approximation introduced in Section 3.3, the renewal analysis in this section is exact.

In the complete-disruption regime, every successful commit disrupts all in-flight attempts that started earlier. Those attempts may continue consuming processing time until their own commit request epochs, but they cannot produce a successful commit. Their later rejected commit requests and subsequent retries are represented through the aggregated attempt rate G. Thus, after each successful commit, the next valid commit must be generated by an attempt that starts after that successful-commit epoch. This regeneration property enables us to derive the complete-disruption throughput map $S(G)$ in the next section.

4.1 Renewal Analysis

In this section, we conduct a renewal analysis to compute the system throughput. Let τ be a successful-commit epoch, and let T be the time from τ to the next successful commit. Under complete disruption, all attempts already in progress at τ are disrupted. Hence, conditioned on the Poisson offered-load approximation, T is determined by the first post-τ attempt that completes. Fig. 4-1 illustrates this renewal structure.

Proposition 1. *Under the Poisson offered-load approximation with $\alpha = 1$, the successful-commit epochs form a renewal process. For every $t \geq 0$,*

$$\Pr(T > t) = \exp\left(-G \int_0^t F_X(u)\, du\right). \tag{4.1}$$

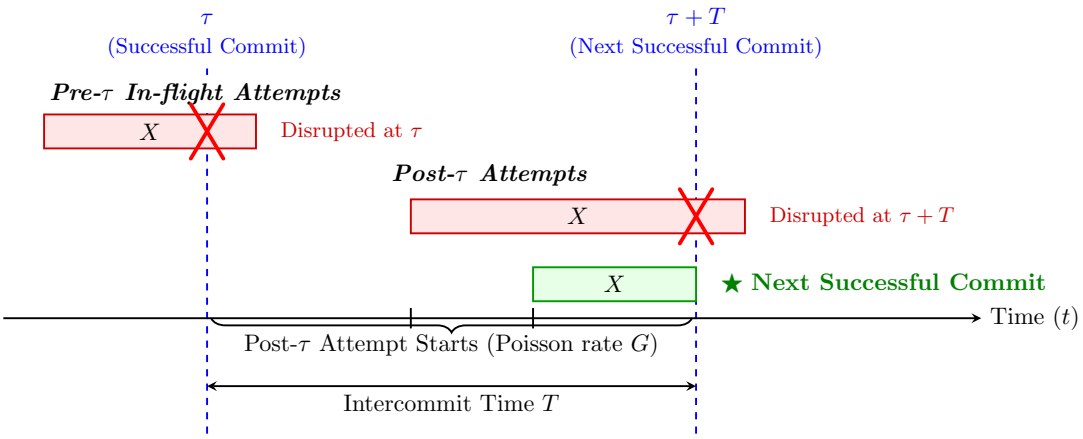


Figure 4-1: Renewal structure in the complete-disruption regime. A successful commit at epoch τ disrupts all in-flight attempts; the next successful commit must therefore be generated by a new attempt started after τ.

Consequently,

$$\mathbb{E}[T] = \int_0^\infty \exp\left(-G\int_0^t F_X(u)\,du\right) dt, \tag{4.2}$$

and

$$\begin{aligned} S(G) &= \left[\int_0^\infty \exp\left(-G\int_0^t F_X(u)\,du\right) dt\right]^{-1}, \\ p_v(G) &= \frac{S(G)}{G}. \end{aligned} \tag{4.3}$$

Proof. Condition on a successful commit at time τ. The event $\{T > t\}$ occurs exactly when no attempt starting in $(\tau, \tau+t]$ finishes by $\tau+t$. An attempt started at time $\tau + t - u$, where $0 \le u \le t$, has probability $F_X(u)$ of finishing by $\tau+t$. Poisson thinning therefore gives a mean measure $G\int_0^t F_X(u)\,du$ for post-τ attempts that finish by $\tau+t$. The void probability gives (4.1). Because the Poisson attempt process has independent increments and all pre-τ attempts have been disrupted, the construction regenerates after each successful commit. Hence $\mathbb{E}[T] = \int_0^\infty \Pr(T > t)dt$, which gives (4.2). The throughput expression follows from the renewal-reward theorem, and the commit-validity probability follows from the rate identity (3.4). □

Proposition 1 shows that, in the complete-disruption regime, the attempt-processing-time distribution enters the throughput through the accumulated completion probability $\int_0^t F_X(u)du$. The following two special cases provide useful benchmarks. Determinis-

tic attempt processing gives the conservative lower-bound case used later in the finite-G analysis, whereas exponential attempt processing illustrates the effect of processing-time variability.

4.1.1 Deterministic Attempt Processing Time

If $X \equiv D$, no new attempt can complete during $(\tau, \tau + D)$. After $\tau + D$, the first attempt started after τ is also the first post-τ attempt that can commit. Thus the inter-successful-commit time is the sum of the deterministic processing time and the waiting time for the first post-τ attempt.

Corollary 2. *For deterministic attempt processing,*

$$\Pr(T > t) = \begin{cases} 1, & t < D, \\ e^{-G(t-D)}, & t \geq D, \end{cases} \tag{4.4}$$

equivalently,

$$T = D + Y, \qquad Y \sim \mathrm{Exp}(G). \tag{4.5}$$

Hence

$$S(G) = \frac{G}{1 + GD}, \qquad p_v(G) = \frac{1}{1 + GD}. \tag{4.6}$$

At a finite operating point,

$$G = \frac{\lambda}{1 - \lambda D}, \qquad p_v = 1 - \lambda D, \qquad \lambda D < 1. \tag{4.7}$$

Proof. For $X \equiv D$,

$$\int_0^t F_X(u)\, du = \begin{cases} 0, & t < D, \\ t - D, & t \geq D. \end{cases}$$

Substituting this expression into (4.1) gives (4.4), which is the survival function of $D + Y$ with $Y \sim \mathrm{Exp}(G)$. Therefore $\mathbb{E}[T] = D + 1/G$, which gives (4.6). Imposing the rate-balance equation $S(G) = \lambda$ gives (4.7). □

4.1.2 Exponential Attempt Processing Time

For exponential attempt processing, the first successful completion after a renewal epoch is not obtained by simply adding an independent processing time to the first post-τ start time, because later attempts may complete earlier than earlier ones. Nevertheless, substituting the exponential distribution into Proposition 1 yields an explicit integral and a closed form.

Corollary 3. *If $X \sim \mathrm{Exp}(1/D)$, so that $F_X(u) = 1 - e^{-u/D}$, then*

$$\Pr(T > t) = \exp\Big(-G\Big[t - D\big(1 - e^{-t/D}\big)\Big]\Big), \qquad t \geq 0, \tag{4.8}$$

and

$$S(G) = \left[\int_0^\infty \exp\Big(-G\Big[t - D\big(1 - e^{-t/D}\big)\Big]\Big)\, dt\right]^{-1}. \tag{4.9}$$

Moreover,

$$S(G) = \frac{1}{D\, e^{GD}(GD)^{-GD}\gamma(GD, GD)}, \tag{4.10}$$

where $\gamma(s,x) = \int_0^x u^{s-1}e^{-u}\,du$ is the lower incomplete gamma function. The corresponding commit-validity probability is

$$p_v(G) = \frac{1}{GD\, e^{GD}(GD)^{-GD}\gamma(GD, GD)}. \tag{4.11}$$

Proof. For $X \sim \mathrm{Exp}(1/D)$,

$$\int_0^t F_X(u)\,du = \int_0^t \big(1 - e^{-u/D}\big)du = t - D\big(1 - e^{-t/D}\big).$$

Substitution into (4.1) gives (4.8), and substitution into (4.2) gives (4.9). To obtain the closed form, use

$$x = e^{-t/D}, \qquad t = -D\ln x, \qquad dt = -Dx^{-1}dx.$$

Then

$$\mathbb{E}[T] = De^{GD}\int_0^1 x^{GD-1}e^{-GDx}\,dx.$$

With $u = GDx$,

$$\int_0^1 x^{GD-1} e^{-GDx}\, dx = (GD)^{-GD} \int_0^{GD} u^{GD-1} e^{-u}\, du.$$

Thus $\mathbb{E}[T] = De^{GD}(GD)^{-GD}\gamma(GD, GD)$, proving (4.10). Equation (4.11) follows from $p_v(G) = S(G)/G$. □

The exponential expression provides a variable-processing-time benchmark for the simulation section. Relative to deterministic processing, short processing-time realizations can produce earlier successful commits and therefore alter the retry amplification required to sustain a given external task-arrival rate.

4.2 Mean Completion Delay

In this section, we derive the mean completion delay. Let W be the completion delay of a typical task in the Poisson offered-load approximation. Also, let N be the number of attempts made by a typical task up to and including its successful attempt. Then the completion time

$$W = \sum_{k=1}^{N} X_k + \sum_{k=1}^{N-1} B_k. \tag{4.12}$$

The first sum is the cumulative processing time of all attempts launched by the task, and the second sum is the cumulative restart overhead after rejected attempts.

At a *finite* operating point G, each external task eventually generates one successful commit. Hence the aggregated attempt rate equals the external task-arrival rate multiplied by the mean number of attempts per task. This leads to

$$G = \lambda\, \mathbb{E}[N], \qquad \mathbb{E}[N] = \frac{G}{\lambda}. \tag{4.13}$$

Since the attempt processing times are assumed to be i.i.d., it then follows from Wald's identity that

$$\mathbb{E}\left[\sum_{k=1}^{N} X_k\right] = \mathbb{E}[X_1]\mathbb{E}[N] = D\,\mathbb{E}[N].$$

Similarly, as the restart delays are assumed to be i.i.d.,

$$\mathbb{E}\left[\sum_{k=1}^{N-1} B_k\right] = \bar{B}\,(\mathbb{E}[N]-1).$$

Combining these identities with (4.13) yields

$$\mathbb{E}[W] = D\frac{G}{\lambda} + \bar{B}\left(\frac{G}{\lambda} - 1\right). \tag{4.14}$$

Equivalently, using the rate-balance equation and the commit-validity identity,

$$\mathbb{E}[W] = \frac{D}{p_v(G)} + \frac{1-p_v(G)}{p_v(G)}\,\bar{B}. \tag{4.15}$$

In particular, for deterministic attempt processing, (4.7) gives

$$\mathbb{E}[W] = \frac{D}{1-\lambda D} + \frac{\lambda D}{1-\lambda D}\,\bar{B}. \tag{4.16}$$

4.2.1 Finite-G Operating Region

Within the Poisson offered-load approximation, an external task-arrival rate λ is in the finite-G operating region if there exists a finite aggregated attempt rate $G^\star < \infty$ satisfying $S(G^\star) = \lambda$. As mentioned before, this can be viewed as a weak stability notion.

A compact sufficient condition follows by lower-bounding the complete-disruption throughput using only the mean attempt processing time. Define

$$H_X(t) \triangleq \int_0^t F_X(u)\,du = \mathbb{E}\left[(t-X)^+\right]. \tag{4.17}$$

Since $(t-x)^+$ is convex in x, Jensen's inequality gives $H_X(t) \geq (t-D)^+$.

Proposition 4. *For any attempt-processing-time distribution with mean D,*

$$S(G) \geq \frac{G}{1+GD}. \tag{4.18}$$

Equality holds for $X \equiv D$.

Proof. Using $H_X(t) \geq (t-D)^+$ in (4.2),

$$\mathbb{E}[T] \leq \int_0^\infty e^{-G(t-D)^+}\,dt = D + \frac{1}{G}.$$

Taking reciprocals proves (4.18). Equality holds when $X \equiv D$. □

Corollary 5. *Under the Poisson offered-load approximation, the complete-disruption regime admits a finite-G operating point whenever*

$$\lambda D < 1. \tag{4.19}$$

Moreover, any finite solution of the rate-balance equation satisfies

$$G \leq \frac{\lambda}{1-\lambda D}. \tag{4.20}$$

Proof. By (4.18),

$$S(G) \geq \frac{G}{1+GD}.$$

The function $G/(1+GD)$ is continuous, equals zero at $G=0$, and increases to $1/D$ as $G \to \infty$. Hence, if $\lambda D < 1$, then there exists a finite G_0 such that

$$\frac{G_0}{1+G_0 D} > \lambda.$$

It follows that

$$S(G_0) \geq \frac{G_0}{1+G_0 D} > \lambda.$$

Since $S(G)$ is continuous and $S(G) \to 0$ as $G \downarrow 0$, the intermediate value theorem implies that there exists some $G^\star \in (0, G_0)$ such that

$$S(G^\star) = \lambda.$$

Thus the rate-balance equation admits a finite-G operating point.

Finally, let G be any finite solution of the rate-balance equation. Then $S(G) = \lambda$. Using

(4.18),

$$\lambda = S(G) \geq \frac{G}{1+GD}.$$

Since $\lambda D < 1$, rearranging gives

$$G \leq \frac{\lambda}{1-\lambda D}.$$

This proves the claim. □

Remark 6. *The condition $\lambda D < 1$ is exact for deterministic attempt processing times and sufficient for general processing-time distributions. It may be conservative when the distribution has substantial mass near short attempt-processing times, because early completions are more likely to win the post-renewal race and thereby increase the successful-commit rate.*

Chapter 5

Performance Analysis for $0 \leq \alpha < 1$

This chapter considers the partial-disruption regime, where $0 \leq \alpha < 1$. Unlike the complete-disruption case $\alpha = 1$, a successful commit does not necessarily invalidate all earlier in-flight attempts.

Fix a successful-commit epoch τ. Throughout this section, a *pre-τ in-flight attempt* refers to an unfinished attempt that started before τ, and a *post-τ attempt* refers to an attempt that starts after τ. In the complete-disruption regime, all pre-τ in-flight attempts are invalidated at τ, so the next successful commit must be generated by a post-τ attempt. This gives the renewal structure used in Chapter 4. For $0 \leq \alpha < 1$, however, a pre-τ in-flight attempt may survive the commit at τ, continue execution, and eventually generate the next successful commit, provided that it is not disrupted by any intervening commit. Thus, the post-τ evolution is no longer determined solely by new attempts started after τ, and the exact renewal structure of Chapter 4 no longer applies.

To analyze this regime, we augment the Poisson offered-load approximation introduced in Section 3.3 with an age-based mean-field approximation. For a prescribed aggregated attempt rate G, the state of the in-flight population is summarized by the ages of the pre-τ in-flight attempts. The number of successful commits encountered by a tagged attempt are then represented by an approximation of a Poisson random variable. The resulting throughput map $S_\alpha(G)$ should therefore be interpreted as the prediction of this mean-field approximation.

5.1 Mean-Field Throughput Analysis

In this section, we develop a mean-field throughput analysis that extends the renewal analysis in Chapter 4.

As described in Section 3.2, every attempt that is in progress at a successful-commit

epoch is invalidated in the complete-disruption regime. However, in the partial-disruption regime $0 \leq \alpha < 1$, some in-flight attempts that started before a successful-commit epoch may remain valid after that commit and may later commit successfully. The throughput analysis must therefore account for the survival of such attempts across multiple successful commits.

Tracking the exact survival history of each in-flight attempt is difficult, because the successful-commit process observed by a tagged attempt is endogenous and statistically coupled with the evolving population of surviving attempts. To obtain a tractable characterization, we introduce a mean-field approximation in which the successful commits observed by a tagged attempt are approximated by a Poisson process with rate S. Under this approximation, the detailed dependence among overlapping attempts is replaced by an average successful-commit field, which is then determined self-consistently through the throughput equations.

Fix a successful-commit epoch τ, and consider a tagged attempt that has age a at time τ. Under the mean-field approximation, the number of successful commits observed by this tagged attempt during the preceding interval of length a, denoted by N_a, is modeled as a Poisson random variable with mean aS. Recall that each such successful commit independently disrupts the tagged attempt with probability α. Conditioned on $N_a = n$, the tagged attempt must survive the n previous successful commits as well as the current successful commit at time τ. Therefore, the probability that a tagged in-flight attempt of age a remains valid immediately after the successful commit at τ is

$$\begin{aligned} r(a;S) &:= \sum_{n=0}^{\infty} (1-\alpha)^{n+1} e^{-Sa} \frac{(Sa)^n}{n!} \\ &= (1-\alpha) e^{-Sa} \sum_{n=0}^{\infty} \frac{((1-\alpha)Sa)^n}{n!} \\ &= (1-\alpha) e^{-\alpha Sa}. \end{aligned} \tag{5.1}$$

For the ease of presentation, we refer to $r(a;S)$ as the *age-dependent retention probability*.

Let T denote the time from a successful-commit epoch τ to the next successful commit under the mean-field approximation. Valid completions in $(\tau, \tau + t]$ come from two disjoint pools: the first pool consists of pre-τ in-flight attempts that survive the commit at τ,

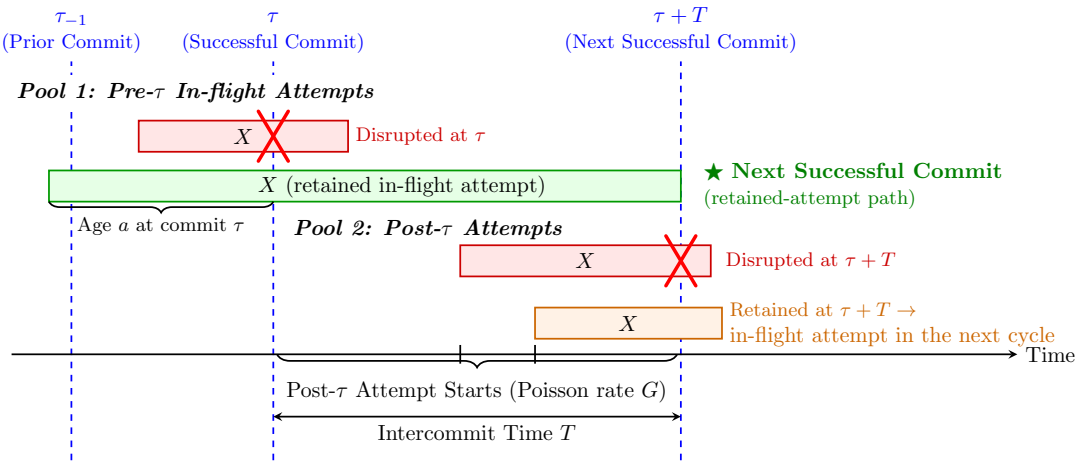


Figure 5-1: Space-time execution timeline for the partial-disruption regime. The next successful commit can arise either from a pre-τ in-flight attempt or from a post-τ attempt. The depicted pre-τ in-flight attempt is schematic: in the mean-field approximation, its retention is age dependent and is represented by $r(a;S)$ in (5.1).

and the second pool consists of post-τ attempts.

Consider first a pre-τ in-flight attempt that started at time $\tau - s$, so that its age at τ is $s > 0$. Such an attempt is still unfinished at τ on the event $X > s$. It completes during $(\tau, \tau + t]$ precisely on the event

$$s < X \leq s + t,$$

whose probability is $F_X(s+t) - F_X(s)$. Under the mean-field approximation, the probability that this age-s attempt remains valid immediately after the commit at τ is represented by the retention probability $r(s;S)$ in (5.1).

Next consider a post-τ attempt. If it starts at time $\tau + t - u$, where $0 \leq u \leq t$, then it has had u units of processing time by $\tau + t$. Hence it completes by $\tau + t$ with probability $F_X(u)$.

Combining these two disjoint sources of candidate completions and using the Poisson thinning argument in the proof of Proposition 1 gives the mean Poisson intensity $GK_\alpha(t;S)$ of candidate completions, where

$$\begin{aligned} K_\alpha(t;S) := & \int_0^\infty r(s;S)\big(F_X(s+t) - F_X(s)\big)\, ds \\ & + \int_0^t F_X(u)\, du. \end{aligned} \tag{5.2}$$

The first term accounts for retained pre-τ in-flight attempts, while the second term accounts for post-τ attempts (as in the renewal analysis). Using the mean-field retention probability

$$r(s;S) = (1-\alpha)e^{-\alpha S s},$$

we can further simplify

$$\begin{aligned} K_\alpha(t;S) := \int_0^\infty (1-\alpha)e^{-\alpha S s}\big(F_X(s+t) - F_X(s)\big)\,ds \\ + \int_0^t F_X(u)\,du. \end{aligned} \tag{5.3}$$

In the following proposition, we characterize the throughput predicted by the mean-field approximation as the solution of a fixed-point equation.

Proposition 7 (Throughput under the age-based mean-field approximation)**.** *Under the Poisson offered-load approximation and the age-based mean-field approximation in* (5.1)*, the intercommit time predicted by the approximation satisfies*

$$\Pr(T > t) = \exp\big(-GK_\alpha(t;S)\big), \qquad t \geq 0. \tag{5.4}$$

Consequently,

$$\mathbb{E}[T] = \int_0^\infty \exp\big(-GK_\alpha(t;S)\big)\,dt, \tag{5.5}$$

and the mean-field successful-commit rate is determined by the fixed-point relation

$$S = \left[\int_0^\infty \exp\big(-GK_\alpha(t;S)\big)\,dt\right]^{-1}. \tag{5.6}$$

Proof. Under the mean-field approximation in (5.3), the number of candidate commits in $(\tau, \tau+t]$ is Poisson with mean $GK_\alpha(t;S)$. The event $\{T > t\}$ is the void event that no such candidate appears, which gives (5.4). Equation (5.5) follows from $\mathbb{E}[T] = \int_0^\infty \Pr(T > t)dt$. Imposing the mean-field consistency condition $S = 1/\mathbb{E}[T]$ gives (5.6). □

The next proposition shows that the fixed point in (5.6) is well posed. Hence, for every prescribed aggregated attempt rate G, the mean-field approximation defines a unique predicted successful-commit rate.

Proposition 8 (Well-posedness of the mean-field fixed point)**.** *Assume $D = \mathbb{E}[X] < \infty$. For every $G > 0$ and $\alpha \in [0,1]$, the mean-field fixed-point equation*

$$S = \Phi_{G,\alpha}(S), \qquad \Phi_{G,\alpha}(S) := \left[\int_0^\infty e^{-GK_\alpha(t;S)}\,dt\right]^{-1} \tag{5.7}$$

admits a unique solution $S_\alpha(G) \in (0,G]$, which defines the mean-field successful-commit rate. Moreover, $G \mapsto S_\alpha(G)$ is continuous and strictly increasing on $(0,\infty)$, and $\lim_{G\downarrow 0} S_\alpha(G) = 0$.

Proof. First, the mapping is well defined. Since the in-flight-attempt term in (5.3) is nonnegative,

$$K_\alpha(t;S) \geq \int_0^t F_X(u)\,du = t - \int_0^t \bar{F}_X(u)\,du \geq (t-D)^+.$$

Therefore $e^{-GK_\alpha(t;S)} \leq e^{-G(t-D)^+}$, and the integral in (5.7) is finite. Conversely, since $(1-\alpha)e^{-\alpha Ss} \leq 1$,

$$\begin{aligned} K_\alpha(t;S) &\leq \int_0^\infty \big(F_X(s+t) - F_X(s)\big)ds + \int_0^t F_X(u)\,du \\ &= \int_0^t \bar{F}_X(u)\,du + \int_0^t F_X(u)\,du = t. \end{aligned}$$

Hence

$$\int_0^\infty e^{-GK_\alpha(t;S)}dt \geq \int_0^\infty e^{-Gt}dt = \frac{1}{G},$$

which implies $0 < \Phi_{G,\alpha}(S) \leq G$.

Next, $K_\alpha(t;S)$ varies continuously with S for each fixed t. Indeed, the only S-dependent term is

$$\int_0^\infty (1-\alpha)e^{-\alpha Ss}\big(F_X(s+t) - F_X(s)\big)ds,$$

and the factor $F_X(s+t) - F_X(s)$ has finite integral

$$\int_0^\infty \big(F_X(s+t) - F_X(s)\big)ds = \int_0^t \bar{F}_X(u)du \leq D.$$

Therefore, for every finite $L > 0$, the integral over $[0,L]$ in (5.7) is continuous in S. Moreover, the bound $e^{-GK_\alpha(t;S)} \leq e^{-G(t-D)^+}$ makes the tail integral over $[L,\infty)$ uniformly small

for all $S \in [0, G]$ as $L \to \infty$. It follows that the improper integral in (5.7) is continuous in S. Since this integral is finite and positive, $\Phi_{G,\alpha}(S)$ is continuous in S.

Since $e^{-\alpha S s}$ is nonincreasing in S, the function $K_\alpha(t;S)$ is nonincreasing in S. Therefore $e^{-GK_\alpha(t;S)}$ and its integral are nondecreasing in S, so $\Phi_{G,\alpha}(S)$ is nonincreasing in S. Hence

$$h_{G,\alpha}(S) = \Phi_{G,\alpha}(S) - S$$

is continuous and strictly decreasing on $[0, G]$. Since $h_{G,\alpha}(0) > 0$ and $h_{G,\alpha}(G) \leq 0$, there is a unique zero in $(0, G]$.

To prove monotonicity in G, fix $0 < G_1 < G_2$. For any fixed S, $K_\alpha(t;S) > 0$ on a set of positive length, and hence

$$\Phi_{G_2,\alpha}(S) > \Phi_{G_1,\alpha}(S).$$

Evaluating this inequality at $S_\alpha(G_1)$ and using the strict decrease of $h_{G_2,\alpha}$ in S gives

$$S_\alpha(G_2) > S_\alpha(G_1).$$

Finally, the same bounded-tail argument shows that $\Phi_{G,\alpha}(S)$ varies continuously with the pair (G, S) for $G > 0$. Since the fixed point is unique, the fixed point $S_\alpha(G)$ must vary continuously with G. Also, $0 < S_\alpha(G) \leq G$, so

$$\lim_{G \downarrow 0} S_\alpha(G) = 0.$$

□

Remark 9. *Proposition 8 provides a direct numerical procedure. For each fixed G, the scalar equation $\Phi_{G,\alpha}(S) - S = 0$ has exactly one solution in $(0, G]$. Hence the mean-field throughput can be computed by a bracketed one-dimensional solve. The rate-balance equation $S_\alpha(G) = \lambda$ can then be solved over G. Since $G \mapsto S_\alpha(G)$ is strictly increasing, this equation admits at most one finite solution under the mean-field approximation.*

5.2 Mean Completion Delay

The partial-disruption regime changes the throughput map used to determine the aggregated attempt rate G, but it does not change the task-level completion-delay analysis. Under the same attempt-renewal assumptions as in Section 4.2, the mean completion delay under the mean-field approximation is

$$\mathbb{E}[W] = D\frac{G}{\lambda} + \bar{B}\left(\frac{G}{\lambda} - 1\right), \tag{5.8}$$

where G denotes the finite operating point satisfying $S_\alpha(G) = \lambda$. Equivalently,

$$\mathbb{E}[W] = \frac{D}{p_{v,\alpha}(G)} + \frac{1 - p_{v,\alpha}(G)}{p_{v,\alpha}(G)}\bar{B}. \tag{5.9}$$

5.2.1 Finite-G Operating Region

Under the mean-field approximation, a finite operating point is a finite $G^\star$ satisfying the rate-balance equation

$$S_\alpha(G^\star) = \lambda,$$

where $S_\alpha(G)$ is the mean-field successful-commit rate in (5.6). In this section, we show that, under the mean-field approximation, every finite external task-arrival rate $0 < \lambda < \infty$ admits a finite-G operating point for any nonnegative attempt-processing-time distribution with finite mean when $0 \le \alpha < 1$.

In the following lemma, we first derive a lower bound for $K_\alpha(t;S)$ in (5.3). We will then use this lower bound to derive an upper bound for the aggregated attempt rate G when the task-arrival rate is λ in Theorem 11.

Lemma 10. *Suppose that $X \ge 0$ has finite mean $\mathbb{E}[X] = D < \infty$. For every $0 \le \alpha < 1$ and $S > 0$,*

$$K_\alpha(t;S) \ge (1-\alpha)e^{-\alpha SD}t, \qquad t \ge 0.$$

Proof. The case $\alpha = 0$ gives $K_0(t;S) = t$, so the claim is immediate. Assume $0 < \alpha < 1$,

and write $a = \alpha S$ and $b = 1 - \alpha$. From the definition of $K_\alpha(t;S)$ in (5.3),

$$K_\alpha(t;S) = \int_0^\infty be^{-as}\big(F_X(s+t) - F_X(s)\big)\,ds + \int_0^t F_X(u)\,du.$$

Using Tonelli's theorem, the first term can be written as

$$\begin{aligned}\int_0^\infty be^{-as}\big(F_X(s+t) - F_X(s)\big)\,ds &= b\,\mathbb{E}\left[\int_{(X-t)^+}^{X} e^{-as}\,ds\right] \\ &= \frac{b}{a}\,\mathbb{E}\left[e^{-a(X-t)^+} - e^{-aX}\right].\end{aligned}$$

Also,

$$\int_0^t F_X(u)\,du = \mathbb{E}\big[(t-X)^+\big].$$

Hence

$$K_\alpha(t;S) = \mathbb{E}\big[(t-X)^+\big] + \frac{b}{a}\mathbb{E}\left[e^{-a(X-t)^+} - e^{-aX}\right].$$

As a function of t, $K_\alpha(t;S)$ is absolutely continuous. For almost every $t \geq 0$,

$$\frac{d}{dt}K_\alpha(t;S) = \mathbb{P}\{X \leq t\} + b\,\mathbb{E}\left[e^{-a(X-t)}\mathbf{1}_{\{X>t\}}\right].$$

Since $b \leq 1$, this implies

$$\begin{aligned}\frac{d}{dt}K_\alpha(t;S) &\geq b\,\mathbb{P}\{X \leq t\} + b\,\mathbb{E}\left[e^{-a(X-t)}\mathbf{1}_{\{X>t\}}\right] \\ &= b\,\mathbb{E}\left[e^{-a(X-t)^+}\right].\end{aligned}$$

By Jensen's inequality, since $x \mapsto e^{-ax}$ is convex,

$$\mathbb{E}\left[e^{-a(X-t)^+}\right] \geq e^{-a\mathbb{E}[(X-t)^+]}.$$

Moreover,

$$\mathbb{E}[(X-t)^+] \leq \mathbb{E}[X] = D.$$

Therefore,

$$\frac{d}{dt}K_\alpha(t;S) \geq be^{-aD} = (1-\alpha)e^{-\alpha SD}$$

for almost every $t \geq 0$. Since $K_\alpha(0;S) = 0$, integration over $[0,t]$ gives

$$K_\alpha(t;S) \geq (1-\alpha)e^{-\alpha SD}t, \qquad t \geq 0.$$

□

Theorem 11. *Suppose that $X \geq 0$ has finite mean $\mathbb{E}[X] = D < \infty$, and let $0 \leq \alpha < 1$. Under the age-based mean-field approximation, for every external task-arrival rate $0 < \lambda < \infty$, the rate-balance equation admits a unique finite solution $G^\star$. Moreover, this solution satisfies*

$$G^\star \leq \frac{\lambda e^{\alpha\lambda D}}{1-\alpha}.$$

Consequently, for every attempt-processing-time distribution with finite mean, the partial-disruption regime has finite-G operating region $(0,\infty)$.

Proof. Fix $0 < \lambda < \infty$ and $0 \leq \alpha < 1$. By Proposition 8, the map $G \mapsto S_\alpha(G)$ is continuous and strictly increasing on $(0,\infty)$, and

$$\lim_{G\downarrow 0} S_\alpha(G) = 0.$$

It remains to show that $S_\alpha(G) \to \infty$ as $G \to \infty$.

At the mean-field fixed point,

$$S_\alpha(G) = \left[\int_0^\infty \exp\{-GK_\alpha(t;S_\alpha(G))\}\,dt\right]^{-1}.$$

By Lemma 10,

$$K_\alpha(t;S_\alpha(G)) \geq (1-\alpha)e^{-\alpha S_\alpha(G)D}t, \qquad t \geq 0.$$

Hence

$$\begin{aligned}\int_0^\infty \exp\{-GK_\alpha(t;S_\alpha(G))\}\,dt &\leq \int_0^\infty \exp\{-G(1-\alpha)e^{-\alpha S_\alpha(G)D}t\}\,dt \\ &= \frac{e^{\alpha S_\alpha(G)D}}{G(1-\alpha)}.\end{aligned}$$

Taking reciprocals gives

$$S_\alpha(G) \geq G(1-\alpha)e^{-\alpha S_\alpha(G)D},$$

or equivalently,

$$S_\alpha(G)e^{\alpha D S_\alpha(G)} \geq G(1-\alpha).$$

Since $1-\alpha > 0$, the right-hand side tends to infinity as $G \to \infty$. Therefore,

$$\lim_{G\to\infty} S_\alpha(G) = \infty.$$

It follows from continuity that, for every $0 < \lambda < \infty$, there exists a finite G such that

$$S_\alpha(G) = \lambda.$$

Strict monotonicity from Proposition 8 further implies that this finite solution is unique.

Finally, at this operating point $S_\alpha(G) = \lambda$. Applying Lemma 10 with $S = \lambda$ gives

$$K_\alpha(t;\lambda) \geq (1-\alpha)e^{-\alpha\lambda D}t.$$

Thus

$$\int_0^\infty \exp\{-GK_\alpha(t;\lambda)\}\,dt \leq \frac{e^{\alpha\lambda D}}{G(1-\alpha)}.$$

Using the rate-balance form

$$\int_0^\infty \exp\{-GK_\alpha(t;\lambda)\}\,dt = \frac{1}{\lambda},$$

we obtain

$$G \leq \frac{\lambda e^{\alpha\lambda D}}{1-\alpha}.$$

This proves both existence of a finite operating point and the stated upper bound. □

Chapter 6

Simulation Results

This section evaluates the analytical results against discrete-event simulations. Unlike the analytical approximation, the simulator explicitly generates external arrivals, retry attempts after rejection, successful commits, and disruptions of overlapping attempts. Thus, the comparison tests whether the proposed Poisson offered-load approximation and the mean-field operating points capture the retry-amplified behavior of the simulated reject-and-retry system.

For $\alpha = 1$, the analytical curves use the complete-disruption throughput formula. For $\alpha < 1$, they use the age-based mean-field approximation. We examine how the external task-arrival rate λ affects the aggregated attempt rate G, the commit-validity probability p_v, and the mean completion delay $\mathbb{E}[W]$.

6.1 Simulation Setup

In our simulations, we consider two attempt-processing-time distributions with mean $D = 1$: deterministic and exponential. The choice $D = 1$ normalizes the time unit, so the external task-arrival rate λ is measured relative to the mean duration of one attempt. These two distributions are used as canonical benchmarks rather than empirical fits to a particular LLM deployment. The deterministic case represents constant-duration or highly predictable workloads, such as fixed-schema data extraction or standardized tool invocations. The exponential case provides a higher-variability benchmark for workloads with variable execution depth, such as open-ended reasoning, iterative code generation, or multi-step web interaction. The restart delay is exponential with mean $\bar{B} = 1$.

The per-successful-commit disruption probability is simulated over $\alpha = 0.1, 0.2, \ldots, 0.9$, 0.99, and 1.0. The figures report the representative values $\alpha \in \{0.2, 0.5, 0.8, 1.0\}$. These plotted values are chosen to show low, moderate, high, and complete disruption while

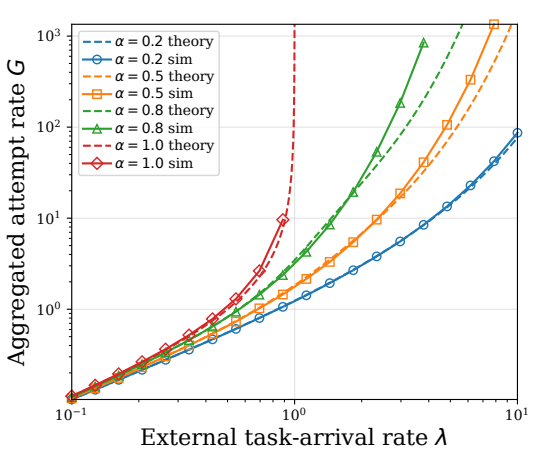
avoiding visual clutter. Smaller values of α represent modular updates or weak dependence among the state components used by overlapping attempts, whereas values near one represent highly coupled shared state or updates that broadly disrupt in-flight execution trajectories.

The external task-arrival rate is varied over a logarithmic grid $\lambda \in [10^{-1}, 10^{1}]$ with 20 sample points. For each parameter setting, external arrivals are generated up to time 10^5, with $[0, 10^4]$ discarded as burn-in. Rate measurements are taken over $[10^4, 10^5]$. Tasks arriving during the measurement interval are tagged for delay measurement and are followed until their first successful commit; if necessary, the simulation continues beyond 10^5 until all tagged tasks complete or the event cap is reached. The maximum number of simulated events is capped at 10^9. Runs that reach this cap are marked incomplete and are excluded from steady-state comparison at that parameter setting.

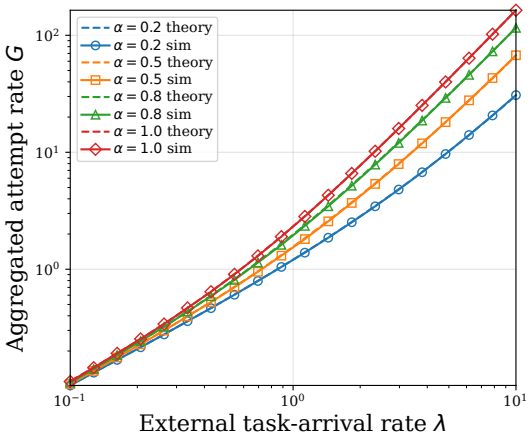
The aggregated attempt rate G is measured as the total attempt rate, including first attempts and retries. The commit-validity probability p_v is measured as the successful-commit rate divided by the aggregated attempt rate. The completion delay W is measured for tasks arriving during the measurement interval, with each such task followed until its first successful commit. Each point is averaged over ten independent seeds. Because the seed-to-seed variability is small relative to the separation among the plotted curves, the figures show seed averages without error bars.

Analytical curves are obtained by solving $S_\alpha(G) = \lambda$, using the renewal formula for $\alpha = 1$ and the mean-field approximation for $\alpha < 1$. The scalar fixed-point equations are solved by bracketed root finding, and the integrals in the throughput expressions are evaluated by numerical quadrature. For the complete-disruption regime, curve termination corresponds to the absence of a finite-G solution. For the partial-disruption regime, curves are reported over the range in which the fixed-point solve remains numerically well conditioned and the corresponding simulation does not exceed the event cap. Simulation runs that exceed the event cap or fall outside this comparable analytical range are not used as steady-state comparison points.

Each performance metric is shown as a two-panel figure, with deterministic processing in panel (a) and exponential processing in panel (b).



(a) Deterministic attempt processing times



(b) Exponential attempt processing times with mean D

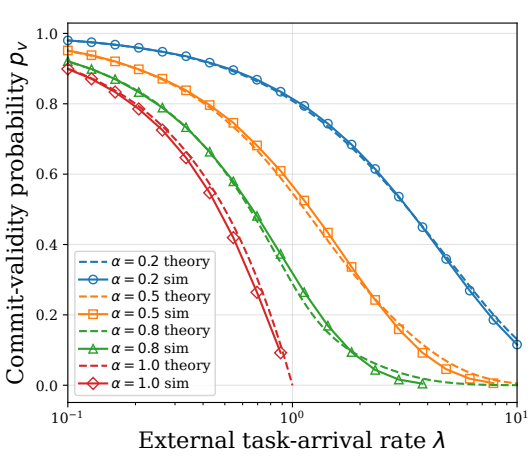
Figure 6-1: Aggregated attempt rate G versus external task-arrival rate λ for (a) deterministic and (b) exponential attempt processing times. Dashed curves show analytical predictions; solid curves with markers show simulation.

6.2 Aggregated Attempt Rate Versus External Task-Arrival Rate

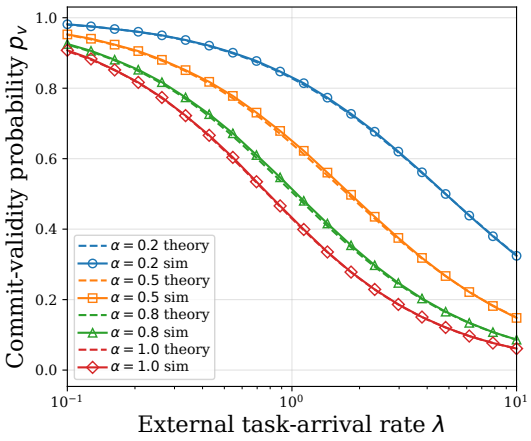
Fig. 6-1 shows the aggregated attempt rate G as a function of the external task-arrival rate λ under deterministic and exponential processing-time distributions. Across both distributions, G increases with λ, and the increase is steeper for larger α. This illustrates the retry-amplification effect: when successful commits disrupt a larger fraction of overlapping attempts, more retry attempts are required to support a given external task-arrival rate. Conversely, smaller α preserves more in-flight attempts and reduces the aggregated attempt rate required at the operating point. The analytical curves closely track the simulation averages and preserve the ordering across α.

6.3 Commit-Validity Probability Versus External Task-Arrival Rate

Fig. 6-2 shows the commit-validity probability $p_v = S/G$. The commit-validity probability decreases as λ increases, because a larger external task-arrival rate induces more



(a) Deterministic attempt processing times



(b) Exponential attempt processing times with mean D

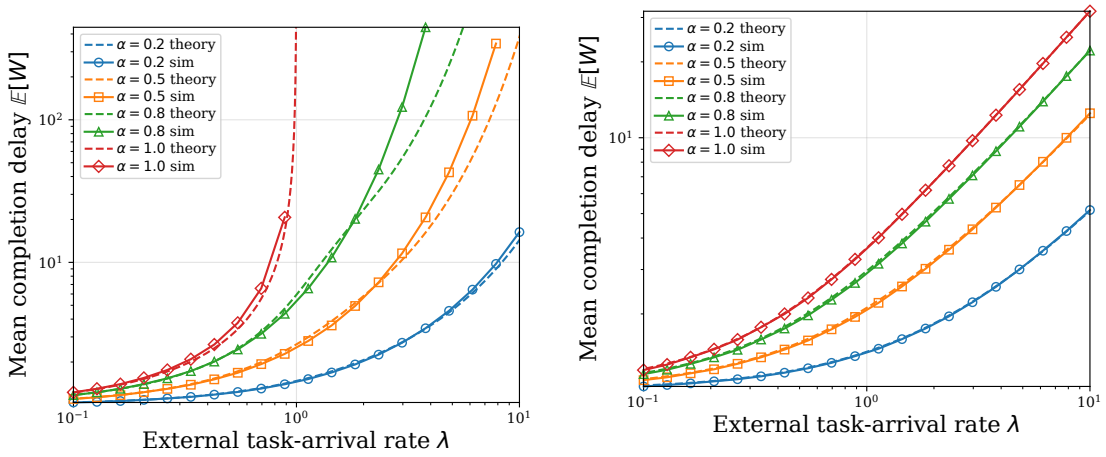
Figure 6-2: Commit-validity probability $p_v = S/G$ versus external task-arrival rate λ for (a) deterministic and (b) exponential attempt processing times. Dashed curves show analytical predictions; solid curves with markers show simulation.

overlapping attempts and therefore greater exposure to intervening successful commits. For fixed λ, smaller α yields larger p_v, since each successful commit disrupts fewer in-flight attempts. This behavior is consistent with the retention mechanism in the partial-disruption analysis.

The exponential processing-time case exhibits a higher commit-validity probability at high load because short processing-time realizations can commit before being exposed to many intervening commits. Thus, processing-time variability can mitigate stale-work loss by allowing some attempts to complete quickly, whereas deterministic attempts remain exposed for a fixed duration.

6.4 Mean Completion Delay Versus External Task-Arrival Rate

Fig. 6-3 shows the mean completion delay $\mathbb{E}[W]$. The delay increases with λ and with α, reflecting the fact that rejected tasks accumulate both additional attempt-processing times and restart delays. The ordering observed for commit-validity probability p_v is therefore reversed for delay: larger α lowers p_v, increases the mean number of attempts



(a) Deterministic attempt processing times

(b) Exponential attempt processing times with mean D

Figure 6-3: Mean completion delay $\mathbb{E}[W]$ versus external task-arrival rate λ for (a) deterministic and (b) exponential attempt processing times. Dashed curves show analytical predictions; solid curves with markers show simulation.

per completed task, and raises $\mathbb{E}[W]$. The exponential case generally has lower delay because short processing-time realizations can commit before being disrupted.

Near high-load boundaries, G and $\mathbb{E}[W]$ become highly sensitive to small changes in commit-validity probability. Therefore, modest approximation error in p_v can produce larger absolute differences in the aggregated attempt rate and completion delay. In the complete-disruption case, when the rate-balance equation has no finite-G solution, the analytical delay curve terminates, corresponding to $\mathbb{E}[W] \to \infty$. In the partial-disruption case, high-load points may instead require aggregated attempt rates that are too large to resolve reliably under the numerical quadrature and finite-horizon simulation limits. Such points are excluded from the steady-state comparison.

6.5 Discussion

The simulations support the interpretation of the model. The comparison focuses not only on the throughput map $G \mapsto S_\alpha(G)$, but also on the induced mapping from the external task-arrival rate to the aggregated attempt rate, commit-validity probability, and mean completion delay:

$$\lambda \mapsto G^\star \mapsto \big(p_{v,\alpha}(G^\star), \mathbb{E}[W]\big).$$

Across both processing-time distributions, the simulations preserve the predicted ordering in α: smaller α reduces retry amplification, improves commit-validity probability, and lowers delay. Agreement is strongest at low and moderate loads, where the independence assumptions underlying the Poisson offered-load approximation and the age-based mean-field approximation are most accurate.

As a compact numerical check, Table 6-1 reports median errors over the comparable operating points. Since p_v is a bounded probability, we use the absolute error

$$|e_{p_v}| = |\widehat{p}_v - p_{v,\mathrm{th}}|.$$

For the positive scale-dependent quantities G and $\mathbb{E}[W]$, which are shown on logarithmic axes, we instead report the multiplicative factor error

$$F_x = \max\left\{\frac{\widehat{x}}{x_{\mathrm{th}}}, \frac{x_{\mathrm{th}}}{\widehat{x}}\right\}, \qquad x \in \{G, \mathbb{E}[W]\}.$$

This metric treats overprediction and underprediction symmetrically and avoids the scale dependence of absolute error.

Table 6-1: Median simulation error for representative values of α.

α	Dist.	$\lvert e_{p_v}\rvert$	F_G	$F_{\mathbb{E}[W]}$
0.2	det.	0.0103	1.02	1.03
0.2	exp.	0.0102	1.01	1.02
0.5	det.	0.0147	1.09	1.13
0.5	exp.	0.0228	1.05	1.08
0.8	det.	0.0126	1.08	1.11
0.8	exp.	0.0190	1.07	1.09
1.0	det.	0.0141	1.02	1.03
1.0	exp.	0.0010	1.00	1.00

The median errors show close typical agreement between simulation and theory across the representative operating regimes. For exponential processing, the median factor errors for G and $\mathbb{E}[W]$ remain close to one for all plotted values of α. For deterministic processing, the median errors are also small, although the larger high-load deviations visible in the figures indicate greater sensitivity near boundary regimes. The complete-disruption case $\alpha = 1$ is governed by the separate complete-disruption expression and again shows close agreement over the comparable range.

Overall, the simulations support the principal analytical conclusion: the per-successful-commit disruption probability α controls retry amplification and thereby determines commit-validity probability and completion delay. Lower α preserves more in-flight attempts and reduces the aggregated attempt rate required to support a given external task-arrival rate, whereas larger α produces stronger retry amplification and larger delay.

Chapter 7

Conclusion

This paper studied stale-work invalidation as a workload amplification mechanism in optimistic shared-state AI-agent systems. When rejected attempts retry, the external task-arrival rate λ is no longer the sole load measure experienced by the system. Instead, the relevant quantity is the aggregated attempt rate $G^\star$, which is selected by the rate-balance equation $S_\alpha(G^\star) = \lambda$. This distinction is central: the same external task-arrival rate can induce substantially different aggregated attempt rates depending on the probability that successful commits disrupt overlapping work.

Under the complete-disruption regime, the Poisson offered-load approximation admits a renewal characterization of successful-commit epochs. This structure yields throughput expressions for general attempt-processing-time distributions and explicit deterministic and exponential special cases. In contrast, under the partial-disruption regime, in-flight attempts may survive intervening successful commits and later complete successfully. This survival precludes the same renewal structure and motivates the proposed age-based mean-field approximation, which captures both attempts that start after a commit epoch and surviving in-flight attempts through a fixed-point characterization.

Within the proposed Poisson offered-load approximation and the age-based mean-field approximation, the per-successful-commit disruption probability α emerges as a central model parameter governing retry amplification, commit-validity probability, and completion delay. Smaller values of α increase the retention of in-flight attempts, reduce the aggregated attempt rate required to support a given external task-arrival rate, and lower completion delay. Larger values of α, especially under high external load, can require a very large aggregated attempt rate and lead to substantial delay. The simulations support these predicted trends and show that the performance impact of invalidation is expressed not only in commit rejection, but also through the retry traffic that rejected attempts generate.

These results suggest that, in distributed multi-agent systems where broad shared-state updates frequently disrupt overlapping work, additional compute capacity alone may be insufficient unless it also reduces attempt durations or changes the invalidation structure. System designs that reduce the effective disruption probability—for example, through finer-grained read/write boundaries, modular shared-state updates, version-aware memory management, or preservation of reusable intermediate work—can reduce retry amplification and improve sustainable throughput.

The present model deliberately abstracts away several important system features, including finite compute-server queueing, structured read/write conflicts, adaptive retry policies, and semantic dependencies among memory regions, tool outputs, and agent-generated updates. Future work should replace the scalar disruption probability with conflict models derived from read/write sets, memory regions, semantic dependencies, and update structure. Further extensions include adaptive backoff and retry control, mechanisms for preserving useful partial execution, and finite-resource models that combine stale-work invalidation with explicit compute-server queueing delay.

參考文獻

[1] H. P. Nii, "Blackboard systems, part one: The blackboard model of problem solving and the evolution of blackboard architectures," *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986.

[2] H. P. Nii, "Blackboard systems, part two: Blackboard application systems, blackboard systems from a knowledge engineering perspective," *AI Magazine*, vol. 7, no. 3, pp. 82–106, 1986.

[3] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, p. 9, 2024.

[4] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, p. 186345, 2024.

[5] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "Mind" exploration of large language model society," in *Advances in Neural Information Processing Systems*, vol. 36. Red Hook, NY, USA: Curran Associates, Inc., 2023, pp. 51 991–52 008.

[6] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 15 174–15 186.

[7] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Proceedings of the First Conference on Language Modeling*. Philadelphia, PA, USA: OpenReview.net, 2024.

[8] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, L. Z. Liu, Y. Xu, H. Su, D. Shin, C. Xiong, and T. Yu, "Openagents: An open platform for language agents in the wild," in *Proceedings of the First Conference on Language Modeling*. Philadelphia, PA, USA: OpenReview.net, 2024.

[9] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. New York, NY, USA: Association for Computing Machinery, 2023, pp. 2:1–2:22.

[10] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model-based agents," *ACM Transactions on Information Systems*, vol. 43, no. 6, pp. 155:1–155:47, 2025.

[11] Z. Tan, J. Yan, I.-H. Hsu, R. Han, Z. Wang, L. Le, Y. Song, Y. Chen, H. Palangi, G. Lee, A. R. Iyer, T. Chen, H. Liu, C.-Y. Lee, and T. Pfister, "In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 8416–8439.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations*. Kigali, Rwanda: OpenReview.net, 2023.

[13] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer,

N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36. Red Hook, NY, USA: Curran Associates, Inc., 2023, pp. 68 539–68 551.

[14] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," in *The Twelfth International Conference on Learning Representations*. Vienna, Austria: OpenReview.net, 2024.

[15] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[16] G. I. Falin, "A survey of retrial queues," *Queueing Systems*, vol. 7, pp. 127–167, 1990.

[17] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*. Vienna, Austria: OpenReview.net, 2024.

[18] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. Jeju, South Korea: International Joint Conferences on Artificial Intelligence Organization, 2024, pp. 8048–8057.

[19] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36. Red Hook, NY, USA: Curran Associates, Inc., 2023, pp. 8634–8652.

[20] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2web: Towards a generalist agent for the web," in *Advances in Neural Information Processing Systems*, vol. 36. Red Hook, NY, USA: Curran Associates, Inc., 2023, pp. 28 091–28 114.

[21] P. A. Bernstein and N. Goodman, "Concurrency control in distributed database systems," *ACM Computing Surveys*, vol. 13, no. 2, pp. 185–221, 1981.

[22] P. A. Bernstein and N. Goodman, "Multiversion concurrency control—theory and algorithms," *ACM Transactions on Database Systems*, vol. 8, no. 4, pp. 465–483, 1983.

[23] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A critique of ANSI SQL isolation levels," in *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. New York, NY, USA: Association for Computing Machinery, 1995, pp. 1–10.

[24] A. Adya, B. Liskov, and P. E. O'Neil, "Generalized isolation level definitions," in *Proceedings of the 16th International Conference on Data Engineering*. Los Alamitos, CA, USA: IEEE Computer Society, 2000, pp. 67–78.

[25] A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, and D. Shasha, "Making snapshot isolation serializable," *ACM Transactions on Database Systems*, vol. 30, no. 2, pp. 492–528, 2005.

[26] D. A. Menascé and T. Nakanishi, "Optimistic versus pessimistic concurrency control mechanisms in database management systems," *Information Systems*, vol. 7, no. 1, pp. 13–27, 1982.

[27] R. J. T. Morris and W. S. Wong, "Performance analysis of locking and optimistic concurrency control algorithms," *Performance Evaluation*, vol. 5, no. 2, pp. 105–118, 1985.

[28] R. Agrawal, M. J. Carey, and M. Livny, "Concurrency control performance modeling: Alternatives and implications," *ACM Transactions on Database Systems*, vol. 12, no. 4, pp. 609–654, 1987.

[29] P. S. Yu, D. M. Dias, and S. S. Lavenberg, "On the analytical modeling of database concurrency control," *Journal of the ACM*, vol. 40, no. 4, pp. 831–872, 1993.

[30] A. Thomasian, "Distributed optimistic concurrency control methods for high-performance

transaction processing," *IEEE Transactions on Knowledge and Data Engineering*, vol. 10, no. 1, pp. 173–189, 1998.

[31] Y. Wu, C.-Y. Chan, and K.-L. Tan, "Transaction healing: Scaling optimistic concurrency control on multicores," in *Proceedings of the 2016 International Conference on Management of Data*. New York, NY, USA: Association for Computing Machinery, 2016, pp. 1689–1704.

[32] M. Dashti, S. B. John, A. Shaikhha, and C. Koch, "Transaction repair for multi-version concurrency control," in *Proceedings of the 2017 ACM International Conference on Management of Data*. New York, NY, USA: Association for Computing Machinery, 2017, pp. 235–250.

[33] J. Guo, P. Cai, J. Wang, W. Qian, and A. Zhou, "Adaptive optimistic concurrency control for heterogeneous workloads," *Proceedings of the VLDB Endowment*, vol. 12, no. 5, pp. 584–596, 2019.

[34] J. R. Artalejo and A. Gómez-Corral, "Analysis of a stochastic clearing system with repeated attempts," *Communications in Statistics. Stochastic Models*, vol. 14, no. 3, pp. 623–645, 1998.

[35] J. R. Artalejo and A. Gómez-Corral, "On a single server queue with negative arrivals and request repeated," *Journal of Applied Probability*, vol. 36, no. 3, pp. 907–918, 1999.

[36] R. J. Boucherie and O. J. Boxma, "The workload in the M/G/1 queue with work removal," *Probability in the Engineering and Informational Sciences*, vol. 10, no. 2, pp. 261–277, 1996.

[37] D. R. Jefferson, "Virtual time," *ACM Transactions on Programming Languages and Systems*, vol. 7, no. 3, pp. 404–425, 1985.

[38] A. Gupta, I. F. Akyildiz, and R. M. Fujimoto, "Performance analysis of Time Warp with multiple homogeneous processors," *IEEE Transactions on Software Engineering*, vol. 17, no. 10, pp. 1013–1027, 1991.

[39] A. Heindl and G. Pokam, "An analytic framework for performance modeling of software transactional memory," *Computer Networks*, vol. 53, no. 8, pp. 1202–1214, 2009.

[40] P. Di Sanzo, B. Ciciani, R. Palmieri, F. Quaglia, and P. Romano, "On the analytical modeling of concurrency control algorithms for software transactional memories: The case of commit-time-locking," *Performance Evaluation*, vol. 69, no. 5, pp. 187–205, 2012.

[41] N. Abramson, "The aloha system—another alternative for computer communications," in *Proceedings of the Fall Joint Computer Conference*, ser. AFIPS Conference Proceedings, vol. 37. New York, NY, USA: Association for Computing Machinery, 1970, pp. 281–285.